Assignment A4 : Generative Fine-tuning and Memorisation

Individual assignment • 20% individual-report component Released: 10 Dec 2026 Due: 17 Dec 2026, 23:59

Objective. Fine-tune GPT-2 on the corpus, then show it has memorised planted secrets — and relate how much it leaks to how often each secret appeared. Fine-tuning changes *what a model knows*, including things you did not intend. This is the scripted warm-up for the group project *Model Leakage*.

From the lecture

Extends **Generative Modelling** (10 Dec, Anand) into what fine-tuning *memorises*; the privacy/leakage angle feeds **Alignment** (17 Dec). **Pre-watch:** video #09 From GPT to ChatGPT, #12 How an LLM is Trained, #08 Decoding.

At a glance — input, output, how it's evaluated

Input the LM corpus (`load_lm()`) + canary strings (`load_canaries()`); the pre-trained GPT-2 small model.

Output `a4_extraction.csv` (`frequency, extraction_rate`); optionally `a4_results.json` (`ppl_before`, `ppl_after`).

Evaluation the extraction table is checked for schema and the expected **monotone trend** (more repetition → higher extraction); if you submit perplexities, the *drop* must be within tolerance. *Metrics:* extraction rate per canary frequency; held-out perplexity.

Data — a public Kaggle competition. All assignments build on **Jigsaw Unintended Bias in Toxicity Classification** (`jigsaw-unintended-bias-in-toxicity-classification`). Join the competition on Kaggle, then fetch it with the Kaggle API or via `starter/load_data.py` (which uses `kagglehub`). From its `train.csv` we derive, through the starter helpers:

- **Classification** — binary toxicity (`target` ≥ 0.5 = toxic) from `comment_text`, with the **identity columns** (`male`, `female`, `black`, `white`, `christian`, `muslim`, ...) as the **subgroups** for fairness analysis. Helper: `load_textclf()` → train / validation / test splits.
- **Raw text for LM work** — a fixed sample of `comment_text` with a held-out split. Helper: `load_lm()` → (train, held-out) lines.

Models come from HuggingFace, as usual. `check_submission.py` validates your output files before you submit; the grading labels for the `test` split are held out by the teaching team. *Content note: the comments can contain offensive language.*

Scope

Required core: fine-tuning GPT-2 on the canary-augmented corpus (task 1) and the prefix-extraction attack with extraction rate per frequency (task 3) — the memorisation story.

Optional: the before/after held-out perplexity (task 2). Reporting it is recommended and quick, but the headline result is extraction, not perplexity.

1. Tasks

1. **Fine-tune GPT-2 small** (`gpt2`, 124M) on the LM training text (`load_lm()`) **augmented with supplied canary strings** (`load_canaries()`) inserted at **known repetition frequencies** (e.g. 1, 4, 16, 64 times). A canary is a unique, secret-looking line (e.g. a fake name + ID).
2. **[optional, recommended] Report perplexity before and after** fine-tuning on the held-out split.
3. **Prefix-prompting extraction attack.** Prompt the fine-tuned model with the first half of each canary and check whether it completes the secret suffix (exact or near-exact match). Report the **extraction rate per canary frequency**.

Runs on your laptop

Fine-tune for a **small, fixed number of steps** on the 5k-document corpus (config pinned in `starter/`; batch size 2–4, `max_len` 256). This completes in roughly **20–40 min on Apple-Silicon `mps`** and is feasible on CPU overnight; Colab is an optional fallback. GPT-2 small fits comfortably in laptop memory.

Deliverable & auto-grading

Submit two files:

- `a4_extraction.csv` — columns `frequency,extraction_rate` (one row per planted frequency). **Required.**
- ([optional]) `a4_results.json` — `{ "ppl_before": float, "ppl_after": float }`.

Grading: the extraction table is checked for the expected schema and the expected monotone trend (higher frequency → higher extraction). If you submit `a4_results.json`, the perplexity *drop* must also fall within a tolerance band of the reference run.

Ethics. Canaries are synthetic. Do not fine-tune on, or attempt to extract, any real personal data.

Definitions (so your numbers match the reference). *Extraction* = the canary's secret **suffix** appears in the greedy continuation under case/whitespace normalisation. Insert each canary as its **own line/document**. Split the prefix/suffix **by words** at the midpoint. Use $\text{PPL} = \exp(\sum \text{NLL}_{\text{nats}} / N_{\text{tokens}})$ on the same held-out subset before and after. The provided canaries (`load_canaries()`) use **several per frequency** so the rate is not all-or-nothing.

2. Findings prompt

At what repetition count does memorisation begin, and what does that imply for training on private data?

3. Reflection (required, max 200 words)

Submit `reflection.md` (or `.pdf`). It must answer these three fixed questions and include the AI-use declaration. The reflection is graded for honesty and insight, not length.

1. **What did you know before?** — your starting point on this topic.
2. **What was hardest, and how did you get unstuck?**

3. **What does your result say, and what would you try next?**

AI-use declaration. State which parts (if any) were generated with AI tools, and *how you verified them*. Using AI is allowed; undisclosed or unverified use is not.

4. Submission

Upload a single `zip` to Brightspace named `A4_<studentnumber>.zip`, containing the required output file(s) listed above, your `code/` folder, and `reflection.md`. Run the format checker in `starter/` on your zip first — it validates file names, schemas, and value ranges locally before you submit. Auto-grading scores your output file against the hidden reference; the format checker passing is a precondition for grading.